

```
> ./Pomotimer CLI
```

resource_path() + imports

```
1 import argparse, time, os, sys, subprocess
2 from tomato_art import TOMATO
3
4 def resource_path(relative_path):
5     base_path = getattr(sys, "_MEIPASS", os.path.abspath("."))
6     return os.path.join(base_path, relative_path)
```

Key Imports

- modules for command-line parsing (`argparse`)
- timing (`time`)
- OS navigation (`os, sys`)
- subprocess execution (`subprocess`)
- ASCII art from `tomato_art` file.

Resource Path Function

`resource_path` function resolves paths to bundled assets, working in both development (uses current directory) and frozen executables (uses PyInstaller's temp `_MEIPASS` dir via `sys._MEIPASS`). It joins a relative path to this base for reliable file access post-packaging.

parse_time() + ANSI escape codes

```
8 RED = "\033[31m"
9 RESET = "\033[0m"
10
11 def print_tomato(color_on=True):
12     if color_on:
13         print(RED + TOMATO + RESET)
14     else:
15         print(TOMATO)
16
17 def parse_time(s):
18     s = s.strip().lower()
19     if not s:
20         raise ValueError("Empty time value")
21
22     if 'h' in s:
23         hours_part, rest = s.split('h', 1)
24         hours = int(hours_part) if hours_part else 0
25         minutes = int(rest) if rest else 0
26         return hours * 60 + minutes
27
28     if s.endswith('m'):
29         return int(s[:-1])
30
31     return int(s)
```

RED and RESET are ANSI escape sequences (`\033[31m` for red text, `\033[0m`

`print_tomato(color_on=True)` outputs the TOMATO ASCII art: in red if `color_on` is True (wrapping with RED + art + RESET),

Parse Time Function

`parse_time(s)` converts shorthand time strings (ex.: "5m", "2h30", "120") to minutes as an integer

- Strips/lowercases input; errors on empty.
- Handles "XhY" -> X*60 + Y minutes.
- Handles "Xm" -> X minutes.
- Defaults to minutes if plain number.

play_sound() + time control functions

```
33 def play_sound():
34     return subprocess.Popen(["mpg123", "-q", "--loop", "2", resource_path("tinker-ring.mp3")])
35
36 def start_timer(minutes, label):
37     total_seconds = minutes * 60
38     for remaining in range(total_seconds, 0, -1):
39         mins, secs = divmod(remaining, 60)
40         print(f"\r{label}: {mins:02d}:{secs:02d}", end="", flush=True)
41         time.sleep(1)
42     print(f"\r{label}: 00:00")
```

start_timer(minutes, label)

Converts `minutes * 60` to `total_seconds`. Uses `range(total_seconds, 0, -1)` countdown loop: computes `divmod(remaining, 60)` for MM:SS, `\r`-overwrites line via `print(..., end="", flush=True)`, then `time.sleep(1)`. Final print clears to "00:00" without `\r`.

set_times()

Prompts for space-delimited input string, splits via `strip().split()` into list `parts`. Returns that as a tuple

UNIX-like help page (argparse)

```
48 def build_parser():
49     parser = argparse.ArgumentParser(
50         prog="pomotimer",
51         description="A simple CLI based pomodoro timer."
52     )
53     parser.add_argument("-w", "--work", default="25", help="work time, e.g. 25 or 1h45")
54     parser.add_argument("-b", "--break-time", default="5", help="short break time, e.g. 5 or 15m")
55     parser.add_argument("-l", "--long-break", default="15", help="long break time, e.g. 15")
56     parser.add_argument("-s", "--sessions", type=int, default=4, help="number of work sessions before the long break")
57     parser.add_argument("--no-color", action="store_true", help="disable colored output")
58     parser.add_argument("--sound", choices=["on", "off"], default="on", help="enable or disable sound")
59     parser.add_argument("-v", "--version", action="version", version="pomotimer 1.0")
60     return parser
```

build_parser()

This `build_parser()` function creates an `argparse.ArgumentParser` configured to auto-generate a Unix-style help page. Supposed to mimic a `man` page (almost) with a formatted usage synopsis, description, and options list with short and long flags. Goal was to make it intuitive for Unix users

Main function logic

```
62 def main():
63     parser = build_parser()
64     args = parser.parse_args()
65
66     color_on = not args.no_color
67     sound_on = args.sound == "on"
68
69     print_tomato(color_on)
70     print("Ctrl+C to exit")
71     print("pomotimer -h for help")
72
73     work_time = parse_time(args.work)
74     break_time = parse_time(args.break_time)
75     long_break_time = parse_time(args.long_break)
76     sessions = args.sessions
```

main()

`main()` starts by parsing CLI args with `build_parser()`, sets `color_on = not args.no_color` and `sound_on = args.sound == "on"`, prints the tomato art + Ctrl+C/help notes, then converts args to minutes with `parse_time()`.

```
78     while True:
79         for i in range(sessions):
80             start_timer(work_time, "Work")
81             if sound_on:
82                 play_sound()
83
84             if i < sessions - 1:
85                 start_timer(break_time, "Break")
86                 if sound_on:
87                     play_sound()
88
89             start_timer(long_break_time, "Long break")
90             if sound_on:
91                 play_sound()
```

While logic

The infinite `while True:` loop drives repeating Pomodoro cycles: `for i in range(sessions)` runs work timer + sound, then if not the final session (`i < sessions - 1`), a short break + sound. Once all sessions are complete, one long break + sound finishes the round before restarting.